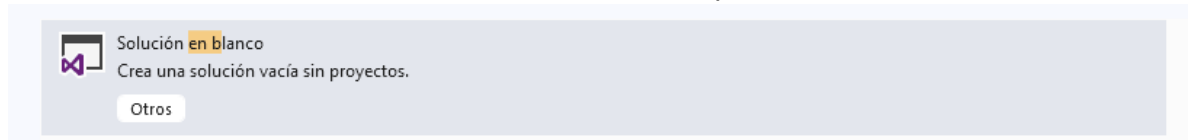


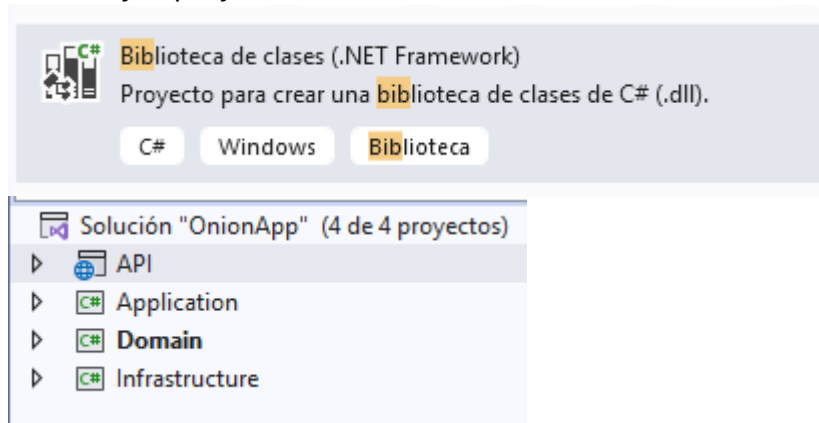
CREACION DE PROYECTO, CON ARQUITECTURA DE LA CEBOLLA.

PASOS.

1. Crear una solución en blanco con el nombre de la aplicación.



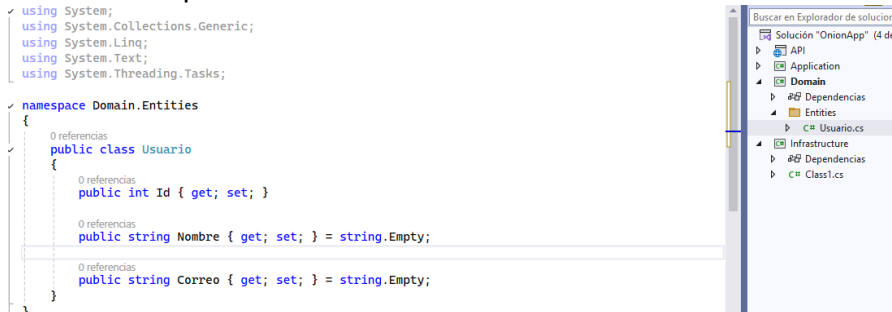
2. Agregar a la solución en blanco las 4 capas en formato bibliotecas de clases de .NET y el proyecto ASP.NET CORE WEB API



3. Agregar las referencias de proyectos de la siguiente manera:

- Api con Application
- Api con Infrastructure
- Infrastructure con Application
- Infrastructure con Domain
- Application con Domain
- Domain no hace ninguna referencia.

4. Creación de primera entidad

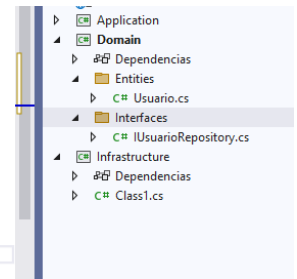


5. Creación de Interfaz para usuario.

```
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Domain.Interfaces
{
    0 referencias
    public interface IUserRepository
    {
        0 referencias
        Task<List<Usuario>> ObtenerTodos();

        0 referencias
        Task<Usuario> Crear(Usuario usuario);
    }
}
```

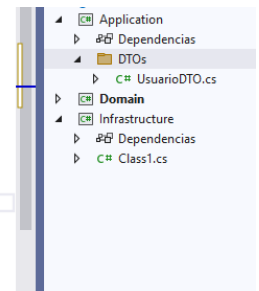


6. Creación de DTO

```
using System.Text;
using System.Threading.Tasks;

namespace Application.DTOs
{
    0 referencias
    public class UsuarioDTO
    {
        0 referencias
        public string Nombre { get; set; } = string.Empty;

        0 referencias
        public string Correo { get; set; } = string.Empty;
    }
}
```



7. Creación de Servicios.

```
0 referencias
public class UsuarioService
{
    private readonly IUserRepository _repository;

    0 referencias
    public UsuarioService(IUserRepository repository)
    {
        _repository = repository;
    }

    0 referencias
    public async Task<List<Usuario>> ObtenerUsuarios()
    {
        return await _repository.ObtenerTodos();
    }

    0 referencias
    public async Task<Usuario> CrearUsuario(UsuarioDTO dto)
    {
        Usuario usuario = new Usuario()
        {
            Nombre = dto.Nombre,
            Correo = dto.Correo
        };

        return await _repository.Crear(usuario);
    }
}
```

8. En la infraestructura instalar los siguientes paquetes.

⬆ Paquetes de nivel superior (3)

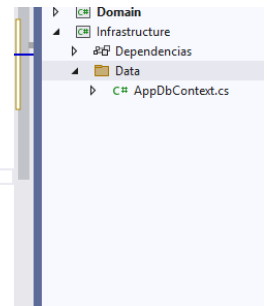
	Microsoft.EntityFrameworkCore por Microsoft	8.0.0
	Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL D...	10.0.8
	Microsoft.EntityFrameworkCore.SqlServer por Microsoft	8.0.0
	Microsoft SQL Server database provider for Entity Framework Core.	10.0.8
	Microsoft.EntityFrameworkCore.Tools por Microsoft	8.0.0
	Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.	10.0.8

9. Creación de clase ApplicationDbContext

```
using System.Linq;
using System.Threading.Tasks;

namespace Infrastructure.Data
{
    2 referencias
    public class ApplicationDbContext : DbContext
    {
        0 referencias
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        0 referencias
        public DbSet<Usuario> Usuarios { get; set; }
    }
}
```



10. Creación del repositorio

```
namespace Infrastructure.Repositories
{
    1 referencia
    public class UsuarioRepository : IUsuarioRepository
    {
        private readonly AppDbContext _context;

        0 referencias
        public UsuarioRepository(AppDbContext context)
        {
            _context = context;
        }

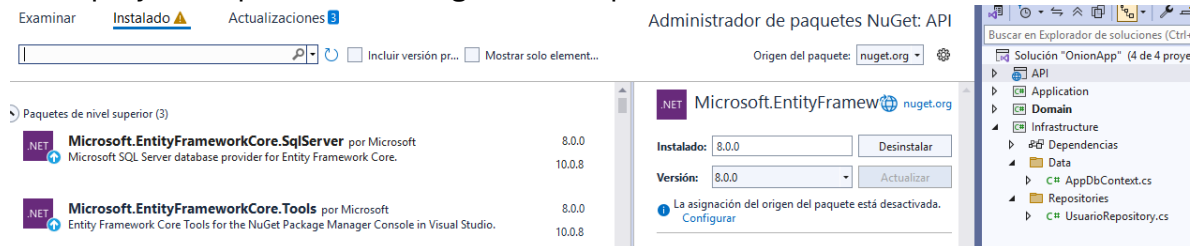
        2 referencias
        public async Task<List<Usuario>> ObtenerTodos()
        {
            return await _context.Usuarios.ToListAsync();
        }

        2 referencias
        public async Task<Usuario> Crear(Usuario usuario)
        {
            _context.Usuarios.Add(usuario);

            await _context.SaveChangesAsync();

            return usuario;
        }
    }
}
```

11. En el proyecto Api instalar las siguientes dependencias.



12. Configurar los servicios de la API.

```
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("conexion")
    );
});

builder.Services.AddScoped<IUsuarioRepository, UsuarioRepository>();

builder.Services.AddScoped<UsuarioService>();
```

13. Creación de controlador y pruebas de funcionamiento.

- Add-Migration Inicial -Project Infrastructure -StartupProject API
- Update-Database -Project Infrastructure -StartupProject API